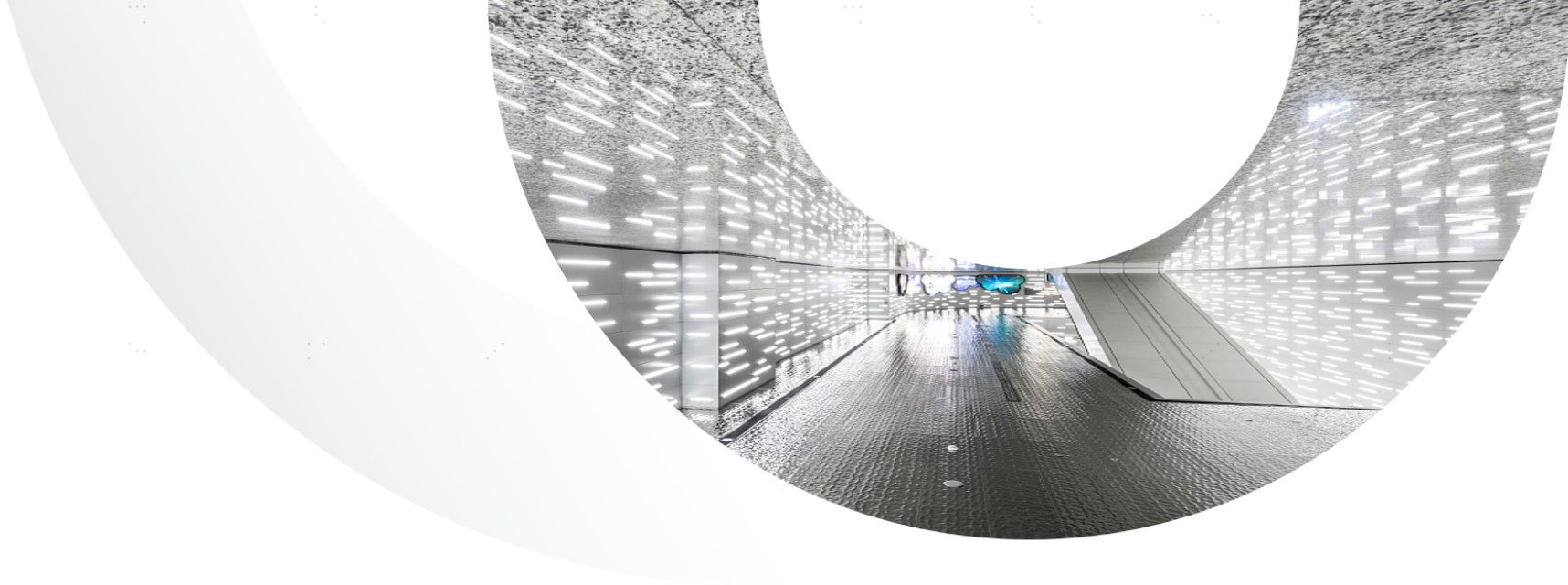


OPPLE



GithubCopilot

□□□□□□□□

超越所见

打造全球化领先照明品牌

GitHub 创始人 Thomas 在 2022 年 6 月 21 日发布了 Github Copilot 开源项目

作者



托马斯·多姆克



在 GitHub，构建让开发人员满意的技术是我们使命的一部分。自去年推出 GitHub Copilot 技术预览版以来，人工智能是为下一代开发人员提供支持的最佳工具之一，这一点已经变得非常清楚。



人工智能已经成为我们日常生活中的副驾驶。它帮助我们撰写电子邮件和文章，自动生成亲人的相册，甚至充当数字助理来帮助我们订购杂货。但到目前为止，人工智能还没有改进代码，软件开发过程几乎完全是手动的。

现在情况正在改变。今天，我很高兴地宣布，我们将向个人开发者全面开放 [GitHub Copilot](#)。你的人工智能配对程序员就在这里。

借助 GitHub Copilot，开发人员可以广泛利用人工智能来编写和完成代码，这在软件历史上尚属首次。就像编译器和开源的兴起一样，我们相信人工智能辅助编码将从根本上改变软件开发的本质，为开发人员提供一种新工具，让他们可以更轻松、更快速地编写代码，从而让他们的生活更加快乐。

@稀土掘金技术社区

GitHub Copilot

GitHub Copilot 使开发人员能够

- 获取基于 AI 的编码建议:

获取与项目上下文和风格约定相匹配的代码建议，
并循环选择不同的选项来决定接受、拒绝或编辑哪些内容。

- **使用您喜欢的环境：**将 GitHub Copilot 与流行的编辑器集成，包括 Neovim、JetBrains IDE、Visual Studio 和 Visual Studio Code 作为不显眼的扩展。

- **在不熟悉的领域自信地编码：**使用新语言编码或尝试新事物，并让 GitHub Copilot 建议数十种语言的语法和代码 - 这样您就可以花更多时间边做边学。

@稀土掘金技术社区

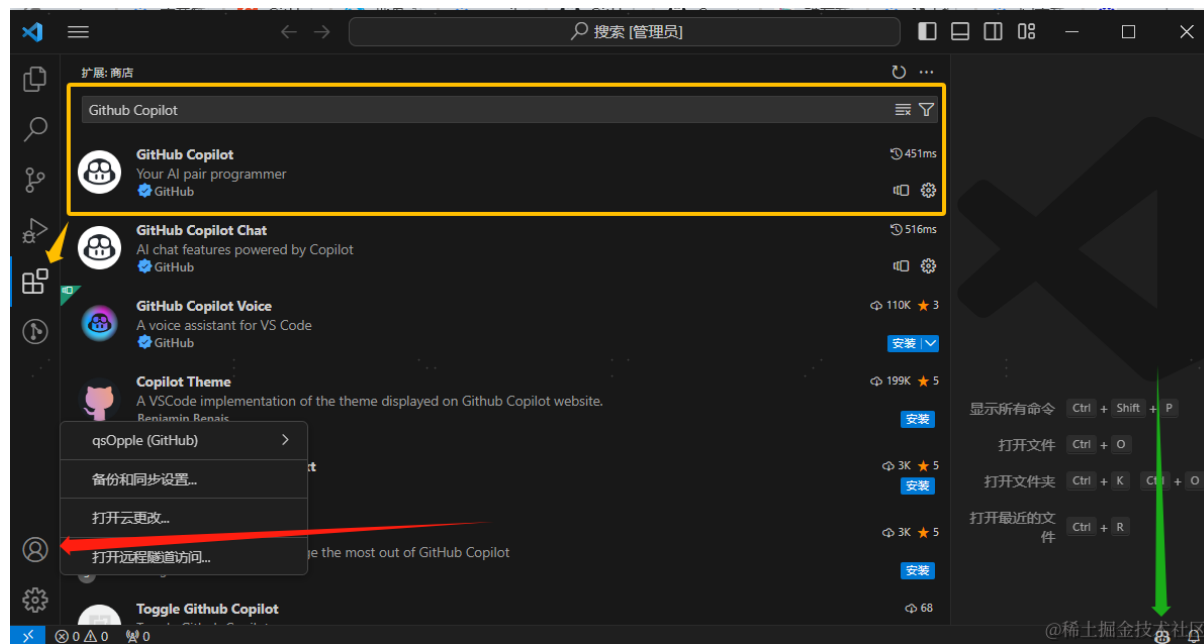
☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐ (copilot) ☐☐☐☐☐

🎉 手把手教你 VS Code 安装

1. 安装 github 扩展

2. 安装 Github Copilot 扩展

3. 安装 Copilot 机器人



Copilot 手把手教你

手把手教你



```
1  <!-- 手把手教你 -->
2  <template>
3    <div class="other">
4      <div class="btn" @click="changeColor">
5        <div class="btn-inner">
6          <div class="btn-inner-inner">手把手</div>
7        </div>
8      </div>
9    </div>
10 </template>
11 <!-- 手把手教你 -->
12 <script>
13 export default {
14   data() {
15     return {
16       color: '#fff'
17     }
18   },
19   methods: {
20     changeColor() {
21       this.color = '#' + Math.floor(Math.random() * 0xf
22       // 手把手教你
23       this.$el.style.backgroundColor = this.color
24     }
25   }
26 }
27 </script>
```

0000

00000000000000000000

- 00000js0000
- 000css00000

```
1 <script setup>
2 import { onMounted } from 'vue';
3 onMounted(() => {
4
5 })
6 // 简单生成一个随机数
7 </script>
8 <template>
9   <div>
10
11   </div>
12 </template>
13 <style scoped></style>
```



Copilot 的 代码 生成 能力 真的 很 强

- 代码 生成 能力 真的 很 强 55% 的 时间
- 代码 生成 能力 真的 很 强 的 代码 生成 能力 (真的 很 强 的 代码 生成 能力 😊)

```
    },
    methods: {
      mouseListener(e) {
        let box = document.querySelector(".text-box").getBoundingClientRect();
        let dt = document.querySelectorAll(".dot");
        for (let i = 0; i < dt.length; i++) {
          //圆灯相对于 容器的坐标位置
          let x = e.pageX - box.left - dt[i].getBoundingClientRect().width / 2;
          let y = e.pageY - box.top - dt[i].getBoundingClientRect().height / 2;
          gsap.to(dt[i],{
            x: x,
            y: y
          });
        }
      }
    }
  };
</script>
```

CopilotChat

如何生成一个垂直居中且背景颜色会随机改变的按钮

qsOpple

帮我校验下代码

GitHub Copilot

已使用 1 引用

这段代码看起来是在 Vue 组件的方法或生命周期钩子中设置了一个 `button` 属性的值。但是，这段代码缺少上下文，所以我无法完全确定它是否正确。在 Vue 组件中，我们通常会在 `data` 函数中声明响应式属性，然后在方法或生命周期钩子中修改它们。

如果 `button` 是在 `data` 函数中声明的，那么这段代码应该是正确的。如果没有，你需要在 `data` 函数中声明它。以下是一个例子：

```
<template>
  <button>{{ button }}</button>
</template>

<script>
export default {
  data() {
    return {
      button: ''
    }
  },
  created() {
    this.button = '我是按钮'
  }
}
</script>
```

在这个例子中，`button` 是在 `data` 函数中声明的，然后在 `created` 生命周期钩子中设置了值。

如何在 Vue 组件中访问和修改响应式属性?

询问 Copilot 或输入 / 以获取命令

src > views > works > cssHtml > other.vue > Vetur > {} "other.vue" > script > default > methods > click

```
1 <!-- 生成一个垂直水平居中的按钮 -->
2 <template>
3   <div class="other">
4     <div class="btn" @click="changeColor">
5       <div class="btn-inner">
6         <div class="btn-inner-inner">按钮</div>
7       </div>
8     </div>
9   </div>
10 </template>
11 <!-- 给按钮添加一个点击事件，每次点击按钮，背景颜色就会随机改变 -->
12 <script>
13 export default {
14   data() {
15     return {
16       color: '#fff'
17     }
18   },
19   methods: {
20     changeColor() {
21       this.color = '#' + Math.floor(Math.random() * 0xffffffff).toString(16)
22       // 背景颜色改变
23       this.$el.style.backgroundColor = this.color
24     },
25     clickButton() {
26       this.button = '我是按钮'
27     }
28   }
29 }
30 </script>
31 <!-- 样式部分 -->
32 <style lang="scss" scoped>
33   .other {
34     width: 100%;
35     height: 100%;
36     display: flex;
37     justify-content: center;
38     align-items: center;
39     height: 100vh;
```

API

API Spring Boot + Mybatis Plus

Copilot Copilot

面试问题

数据库相关 SQL 数据库 Spring Boot + Mybatis Plus

Copilot 数据库 Copilot

--	--	--	--	--	--	--	--	--

□ □ □ □ □ □ □ □ □ :

1.

GitHub Copilot

RESTFUL API

Copilot Copilot

“GET” Copilot

```
// 创建一个GET请求函数，用于返回所有用户数据
@GetMapping("/all")
public List<User> getAllUsers() {
    return userService.list();
}
```

面试题

2. 更新用户年龄

使用 Copilot 生成更新用户年龄的代码

生成更新用户年龄的代码

使用 Copilot 生成更新用户年龄的代码
使用 Copilot 生成更新用户年龄的代码

// 更新用户集合并返回更新列数，使用冒泡排序只更新年龄最大的前三个用户

```
public R<Integer> updateUserList(List<User> userList) {  
    int result = 0;  
    for (int i = 0; i < userList.size() - 1; i++) {  
        for (int j = 0; j < userList.size() - 1 - i; j++) {  
            if (userList.get(j).getAge() < userList.get(j + 1).getAge()) {  
                User temp = userList.get(j);  
                userList.set(j, userList.get(j + 1));  
                userList.set(j + 1, temp);  
            }  
        }  
    }  
    for (int i = 0; i < 3; i++) {  
        result += userMapper.updateById(userList.get(i));  
    }  
    return new R<>(result);  
}
```



3. 发送邮件

Copilot

Java

Copilot

```
// 使用MybatisPlus的LambdaQueryWrapper来查询用户集合 条件为用户类型为1 且名称中包含指定字符串的用户集合
// 获取这些用户的邮箱并使用hutool进行发送 发送主题为当前人名称 发送内容为该用户所有信息的拼接字符串

public R<Boolean> sendEmail(String name) {
    List<User> userList = userMapper.selectList(new LambdaQueryWrapper<User>()
        .eq(User::getUserType, val: 1)
        .like(User::getName, name));
    List<String> emailList = userList.stream().map(User::getEmail).collect(Collectors.toList());
    String content = userList.stream().map(User::toString).collect(Collectors.joining( delimiter: "\n"));
    MailUtil.send(emailList, name, content, isHtml: false);
    return new R<>( data: true);
}
```



4. 测试用例

测试用例 Copilot 测试用例测试用例测试用例测试用例

测试用例测试用例测试用例测试用例测试用例测试用例

测试用例 Copilot 测试用例测试用例测试用例测试用例“测试用例” Copilot 测试用例

测试用例测试用例测试用例测试用例测试用例测试用例测试用例测试用例

```
private Boolean isEven(int number) {  
    if (number == 1) return false;  
    else if (number == 2) return true;  
    else if (number == 3) return false;  
    else if (number == 4) return true;  
    else if (number == 5) return false;  
    else if (number == 6) return true;  
    else return false;  
}  
  
// 优化上面的函数  
private Boolean isEven2(int number) {  
    if (number % 2 == 0) return true;  
    else return false;  
}
```

THANKS

